



Implementazione parallela dell'algoritmo di Thomas

Francesco Schirinzi

Contents

1	Introduzione	3
2	Design parallelo	5
2.1	Partizionamento dell'input	6
2.2	Thomas_v1	8
2.3	Gather	10
2.4	Risoluzione con Thomas sequenziale	11
2.5	Scatter	11
2.6	Risoluzione variabili interne	12
2.7	Analisi Asintotica	12
2.8	Scalabilità del sistema parallelo	13
2.9	Modello Analitico	14
2.9.1	Latenza (Λ)	14
2.9.2	Bandwidth (β)	15
2.9.3	Modello analitico	16
3	Utilizzo del programma	18
3.1	Utilizzo osu-micro-benchmarks	18
4	Analisi delle prestazioni e della scalabilità	19
4.1	Input Size (N)	19
4.2	Modello analitico	20
4.3	Speedup ed efficienza	20

4.4	Strong scalability	22
4.5	Weak scalability	23

1 Introduzione

In questo lavoro viene presentata la versione parallela dell'algoritmo di Thomas utilizzata per la risoluzione di sistemi lineari tridiagonali. Per sistema tridiagonale si intende un sistema che assume la seguente forma:

$$\begin{bmatrix} b_1 & c_1 & & & 0 \\ a_2 & b_2 & c_2 & & \\ & a_3 & b_3 & \ddots & \\ & & \ddots & \ddots & c_{n-1} \\ 0 & & & a_n & b_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} d_1 \\ d_2 \\ d_3 \\ \vdots \\ d_n \end{bmatrix}$$

Figure 1: Sistema tridiagonale [4]

cioè un sistema in cui solo gli elementi della diagonale principale e delle due diagonali immediatamente adiacenti, superiore ed inferiore, sono $\neq 0$. La versione sequenziale dell'algoritmo si compone in 2 fasi:

- **Passo di Forward:** In questo passo si eliminano progressivamente gli elementi al di sotto della diagonale principale, per ottenere un sistema triangolare superiore, come illustrato in figura 2.

$$\begin{bmatrix} b_1 & c_1 & & & 0 \\ & b_2 & c_2 & & \\ & & b_3 & \ddots & \\ & & & \ddots & c_{n-1} \\ 0 & & & & b_n \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} d_1 \\ d_2 \\ d_3 \\ \vdots \\ d_n \end{bmatrix}$$

Figure 2: Sistema dopo forward pass

L'idea è quella di applicare una semplificazione dell'algoritmo di Eliminazione di Gauss, in cui non si applica la procedura di eliminazione degli elementi al di sotto della diagonale principale per tutte le righe successive alla riga i , ma solo per la riga successiva. La correttezza del passo di forward è dimostrabile per via della proprietà della matrice tridiagonale che, per costruzione, ha solo l'elemento al di sotto della diagonale $\neq 0$, mentre tutte le altre entry della colonna sono nulle.

In generale, la riga i -esima viene aggiornata secondo la seguente formula:

$$R_i = R_i - w \cdot R_{i-1}$$

dove

$$w = \frac{A[i]}{B[i-1]}$$

- **Passo di Backward:** Una volta ottenuto il sistema triangolare superiore, si procede alla risoluzione partendo dall'ultima incognita x_n che può essere calcolata immediatamente. Successivamente, si determinano in modo iterativo tutte le altre incognite all'indietro. In questa fase si applica una versione semplificata dell'algoritmo di Back Substitution, in quanto, grazie alla struttura tridiagonale della matrice, dopo aver trovato x_i , bisogna aggiornare solo la riga immediatamente precedente, poichè è l'unica che contiene l'incognita x_i appena determinata.

La procedura implementativa dell'algoritmo può essere riassunta dal seguente pseudocodice:

```

-----
                        Algoritmo di Thomas
-----
Input: N, A[0,N-1], B[0,N-1], C[0,N-1], D[0,N-1], X[0,N-1]

// Forward Pass
for i = 1 to N-1:
    w = A[i]/B[i-1]
    B[i] = B[i] - w * C[i-1]
    D[i] = D[i] - w * D[i-1]

// Backward Pass
X[N-1] = D[N-1]/B[N-1]

for i = N-2 to 0:
    x[i] = (D[i] - C[i]*X[i+1]) / B[i]
-----

```

Figure 3: Pseudocodice Thomas sequenziale

A differenza dell'algoritmo di eliminazione di Gauss, che richiede $O(n^3)$ operazioni, l'algoritmo di Thomas ha complessità computazionale pari a $O(n)$.

In figura 3 è importante notare che tale algoritmo, anche se definito per risolvere problemi in forma matriciale, non memorizza la matrice completa

(in quanto è molto sparsa) ma utilizza solo 4 vettori A , B , C , D (sottodiagonale, diagonale principale, diagonale superiore e vettore dei termini noti), risparmiando così una notevole quantità di memoria.

Infine, è importante ricordare che tale algoritmo non gode di stabilità numerica nel caso generale, ma questa può essere dimostrata nei casi in cui la matrice è diagonalmente dominante o definita positiva simmetrica.

2 Design parallelo

Dall'analisi dello pseudocodice presente in figura 3 emerge che sia la fase di forward che quella di backward hanno forti dipendenze tra iterazioni successive: ogni passo utilizza i risultati ottenuti nella riga precedente. Questa caratteristica rende l'algoritmo difficile da parallelizzare.

Una possibile strategia di parallelizzazione consiste nell'utilizzare una variante dell'algoritmo di Thomas che non mira ad eliminare gli elementi al di sotto della diagonale principale, ma a riscrivere le equazioni interne al sistema in funzione delle sole incognite di bordo, eliminando così la dipendenza dalle variabili adiacenti. In questo modo è possibile ridurre il sistema originale a un piccolo sistema ridotto che coinvolge esclusivamente tali incognite. Una volta determinate queste ultime, mediante la risoluzione del sistema ridotto, le variabili interne possono essere calcolate immediatamente, poiché sono state espresse in funzione delle variabili di bordo.

La progettazione parallela dell'algoritmo di Thomas segue i seguenti passi:

1. **Partizionamento dell'input:** I quattro vettori di input, ciascuno di dimensione n , vengono suddivisi tra i p processi disponibili. Ad ogni processo sono quindi assegnati $\frac{4n}{p}$ elementi;
2. **Thomas_v1:** Ogni processo applica la variante dell'algoritmo di Thomas sul blocco di dati assegnato, costruendo un sistema le cui variabili interne dipendono esclusivamente dalle variabili di bordo.
3. **Gather:** Ciascun processo invia al processo P_0 le equazioni di bordo (prima e ultima riga) del proprio sistema locale; in questo modo il processo di rango 0 costruisce un sistema ridotto globale composto da $2p$ equazioni. Questa comunicazione avviene mediante l'utilizzo della `MPI_Gather`;
4. **Risoluzione con Thomas sequenziale:** Il processo P_0 risolve il sistema ridotto utilizzando l'algoritmo di Thomas sequenziale (in quanto il sistema ridotto è un sistema tridiagonale). In questo modo si ottengono i valori delle variabili di bordo per ciascun sistema locale;

5. **Scatter:** Una volta calcolate tali incognite, il processo P_0 le distribuisce ai processi corrispondenti. Tale fase di comunicazione è realizzata tramite una MPI_Scatter;
6. **Risoluzione variabili interne:** Ogni processo ha i valori delle proprie variabili di bordo e un sistema locale in cui ciascuna equazione interna è espressa esclusivamente in funzione di tali variabili. Le incognite interne possono quindi essere calcolate in modo diretto applicando la seguente formula di aggiornamento:

$$X_i = D_i - A_i X_0 - C_i X_{\frac{N}{p}-1}$$

dove X_0 e $X_{\frac{N}{p}-1}$ rappresentano rispettivamente le variabili di bordo superiore ed inferiore del sistema locale, determinate in precedenza tramite la risoluzione del sistema ridotto.

L'intera procedura può essere riassunta di seguito:

```

-----
                        Algoritmo di Thomas Parallelo
-----

1) Distribuire l'input tra i P processi.
2) Ogni processo applica thomas_v1 al suo blocco di dati.
3) Gather: Ogni processo manda la sua prima e ultima riga al
   processo con rango 0.
4) Il processo P0 risolve il sistema ridotto con Thomas sequenziale.
5) Scatter: il processo con rango 0 invia le variabili di
   bordo ai processi corrispondenti.
6) Ogni processo calcola le variabili interne conoscendo le
   variabili di bordo:
   for i = 1 to (N/p - 2) {
       X[i] = D[i] - A[i] * X[0] - C[i] * X[N/p - 1];
   }
-----

```

Figure 4: Progettazione Thomas Parallelo

Nelle sezioni seguenti ciascuna delle fasi descritte in precedenza viene analizzata con un maggiore livello di dettaglio:

2.1 Partizionamento dell'input

In questa sezione viene descritta la strategia adottata per la distribuzione dell'input tra i vari processi. Se n è multiplo di p , ogni processo riceverà

esattamente $\frac{n}{p}$ elementi, in caso contrario, ad alcuni ne verranno assegnati $\left\lceil \frac{n}{p} \right\rceil$ mentre ad altri $\left\lfloor \frac{n}{p} \right\rfloor$.

In quest'ultimo caso, i blocchi di dimensione maggiore vengono distribuiti in modo uniforme tra i processi, evitando che siano assegnati esclusivamente ai primi.

Per effettuare una distribuzione equa bisogna utilizzare le seguenti formule che permettono di:

- Individuare il primo elemento controllato dal processo i :

$$\text{BLOCK_LOW}(i) = \left\lfloor \frac{i \cdot n}{p} \right\rfloor,$$

- Individuare l'ultimo elemento controllato dal processo i :

$$\text{BLOCK_HIGH}(i) = \left\lfloor \frac{(i+1) \cdot n}{p} \right\rfloor - 1$$

- Individuare il j -esimo elemento controllato dal processo i :

$$\text{BLOCK_OWNER}(j) = \left\lfloor \frac{p \cdot (j+1) - 1}{n} \right\rfloor$$

Utilizzando queste formule, i blocchi di taglia più grande sono distribuiti tra i processi. In particolare si può sempre affermare che il primo processo riceve $\left\lceil \frac{n}{p} \right\rceil$ elementi mentre l'ultimo ne riceve $\left\lfloor \frac{n}{p} \right\rfloor$.

Per implementare la partizione dei dati di input è possibile seguire una delle seguenti 2 strategie:

- **Strategia 1:** L'ultimo processo legge dal file il primo elemento, che indica la dimensione del vettore e lo invia in broadcast a tutti gli altri. In questo modo ogni processo può allocare la memoria necessaria per ricevere la parte di dati. Successivamente, l'ultimo processo legge l'intero vettore da file e lo distribuisce a tutti gli altri tramite una MPI_Scatter;
- **Strategia 2:** L'ultimo processo legge il primo elemento del file e lo invia in broadcast a tutti gli altri processi, permettendo loro di allocare la memoria necessaria. Successivamente legge dal file il blocco di dati corrispondente al processo i e glielo invia utilizzando una comunicazione point to point. La lettura da file viene effettuata dall'ultimo processo

in quanto, per costruzione, gestisce $\left\lceil \frac{n}{p} \right\rceil$ elementi, quindi ha sempre la memoria necessaria per leggere il blocco di dati dal file.

Sebbene la prima strategia comporti una minore latenza complessiva ($\lambda \log p + \frac{N \cdot (P-1)}{\beta \cdot P}$ per Scatter e $(P-1) \left(\lambda + \frac{N}{P\beta} \right)$ per la Point to Point), si preferisce la seconda in quanto la distribuzione dell'input non influisce sulle prestazioni dell'algoritmo e, nel caso di vettori molto grandi, leggere l'intero contenuto in memoria potrebbe non essere possibile. Al contrario, la seconda strategia permette di leggere l'input a blocchi risolvendo il problema appena discusso.

2.2 Thomas_v1

Questa variante dell'algoritmo di Thomas ha l'obiettivo di rendere ogni equazione interna dipendente soltanto dalle due variabili di bordo.

In figura 5 è possibile visualizzare lo pseudocodice della variante Thomas_v1.

```

-----
                        Algoritmo di Thomas V1
-----
Input: N, A[0,N-1], B[0,N-1], C[0,N-1], D[0,N-1]

D[0] = D[0]/B[0]
C[0] = C[0]/B[0]
A[0] = A[0]/B[0]

D[1] = D[1]/B[1]
C[1] = C[1]/B[1]
A[1] = A[1]/B[1]

for i = 2 to N-1:
    w = A[i]/B[i-1]
    B[i] = B[i] - w * C[i-1]
    D[i] = D[i] - w * D[i-1]
    A[i] = - w * A[i-1]

    // Normalizzazione riga i per B[i]

for i = N-3 to 0:
    if (i == 0):
        B[i] = B[i] - w * A[i + 1];
        D[i] = (D[i] - w * D[i + 1]) / B[i];
        A[i] = A[i] / B[i];
        C[i] = - (w * C[i + 1]) / B[i];
        B[i] = 1.0;
    else:
        w = C[i] / B[i + 1];
        D[i] = D[i] - w * D[i + 1];
        A[i] = A[i] - w * A[i + 1];
        C[i] = - w * C[i + 1];
-----

```

Figure 5: Pseudocodice Thomas_v1

In fase di inizializzazione viene effettuata la normalizzazione delle prime due righe della matrice rispetto all'elemento B_i . Nel primo ciclo `for` si effettua il passo di forward simile a quello dell'algoritmo di Thomas sequenziale, con l'obiettivo di eliminare l'elemento al di sotto della diagonale principale. In questa variante, però, l'aggiornamento parte dalla terza riga e questo permette di riscrivere ogni riga interna in funzione della variabile di bordo superiore.

Si consideri, ad esempio, la matrice in figura 6:

4.57244	3.57244	0	0	0		-5.17164
-7.36245	9.88978	1.52733	0	0		-3.3916
0	-1.87419	8.62249	-5.7483	0		-2.48146
0	0	-6.77064	16.9584	9.1878		-8.39836
0	0	0	1.52402	2.52402		-0.305652

Figure 6: Matrice Tridiagonale iniziale

Dopo il primo ciclo `for` la matrice assume la forma mostrata in figura 7:

1	0.781298	0	0	0		-1.13104
-0.74445	1	0.154435	0	0		-0.34294
-0.156559	0	1	-0.645012	0		-0.350563
-0.0841853	0	0	1	0.729695		-0.855503
0.0908673	0	0	0	1		0.706931

Figure 7: Matrice dopo 1° ciclo `for`

Successivamente viene applicato un secondo ciclo `for` che parte dalla terzultima riga e ha come obiettivo quello di eliminare l'elemento al di sopra della diagonale principale. L'aggiornamento della riga avviene con la seguente formula di aggiornamento:

$$R_i = R_i - w \cdot R_{i+1}, \quad \text{con } w = \frac{C_i}{B_{i+1}}$$

Partendo dalla terzultima riga, e applicando la formula di aggiornamento oltre a rimuovere l'elemento al di sotto della diagonale principale ogni riga viene riscritta in funzione della variabile di bordo inferiore.

Dopo questo ulteriore passo la situazione è la seguente:

1	0	0	0	0.0364928		-0.624591
-0.711886	1	0	0	-0.0726867		-0.203582
-0.210859	0	1	0	0.470662		-0.902372
-0.0841853	0	0	1	0.729695		-0.855503
0.0908673	0	0	0	1		0.706931

Figure 8: Matrice dopo 2° ciclo `for`

Al termine, le equazioni interne non dipendono più dalle incognite vicine, ma soltanto da quelle di bordo.

2.3 Gather

Dopo aver applicato `Thomas_v1` è necessaria una `MPI_Gather` per comunicare prima e ultima riga del sistema locale al processo P_0 . Questa comunicazione può essere implementata utilizzando 2 strutture di comunicazione differenti:

1. **Grafo completo:** Utilizzando la struttura di comunicazione a grafo completo vengono effettuati $p - 1$ passi di comunicazione e vengono inviati in totale $6(p - 1)$ elementi (2 di **A**, 2 di **C** e 2 di **D**, **B** non viene comunicato in quanto, per via della normalizzazione, è pari ad 1).

Dunque il tempo speso per la comunicazione utilizzando una struttura di comunicazione a grafo completo è:

$$(p - 1) \cdot \left(\lambda + \frac{6}{\beta} \right)$$

2. **Ipercubo:** Utilizzando questa struttura il numero di passi di comunicazione da effettuare è $\log p$ e il numero di messaggi inviati complessivamente è

$$6 \cdot \left(\overbrace{1 + 2 + 2^2 + \dots + 2^{k-1}}^{2^k - 1} \right) = 6(p - 1)$$

con $k = \log p$

Pertanto il costo `MPI_Gather` è

$$\lambda \log p + \frac{6(p - 1)}{\beta}$$

Le due comunicazioni richiedono lo stesso numero totale di messaggi inviati; tuttavia, utilizzando una struttura di comunicazione ad ipercubo, la trasmissione avviene in $\log p$ passi, riducendo la latenza.

2.4 Risoluzione con Thomas sequenziale

In questa sezione il processo P_0 dopo aver ricevuto la prima ed ultima riga dei sistemi locali degli altri processi, crea un sistema ridotto di dimensione $2p$ e lo risolve utilizzando l'algoritmo di Thomas sequenziale.

2.5 Scatter

Anche in questo caso è opportuno scegliere una struttura di comunicazione efficiente, in modo da ridurre il costo legato alla latenza. La `MPI_scatter` può essere implementata nei seguenti 2 modi:

1. **Grafo completo:** Utilizzando la struttura di comunicazione a grafo completo vengono effettuati $p - 1$ passi di comunicazione e vengono inviati complessivamente $2(p - 1)$ elementi (P_0 invia ad ogni processo le 2 variabili di bordo corrispondenti).

Il tempo totale di comunicazione in questo caso è:

$$T_{scatter_grafo} = (p - 1) \cdot (\lambda + \frac{2}{\beta})$$

2. **Ipercubo:** In questo caso il numero di passi di comunicazione da effettuare si riduce a $\log p$. Il numero complessivo di messaggi inviati è:

$$\begin{aligned} \sum_{i=1}^{\log p} \left(\frac{2p}{2^i \beta} \right) &= \sum_{i=1}^{\log p} \frac{2p}{2^i \beta} \\ &= \frac{2p}{\beta} \sum_{i=1}^{\log p} \frac{1}{2^i} \\ &= \frac{2p}{\beta} \left(1 - \frac{1}{2^{\log p}} \right) \\ &= \frac{2(p - 1)}{\beta}. \end{aligned}$$

Il tempo totale di comunicazione in questo caso è:

$$\lambda \log p + \frac{2(p - 1)}{\beta}$$

Anche in questo caso viene scelta la seconda strategia in quanto permette di ridurre la componente legata alla latenza.

2.6 Risoluzione variabili interne

Ogni processo, dopo aver ricevuto le variabili di bordo, può calcolarsi le sue variabili interne con la formula:

$$X_i = D_i - A_i X_0 - C_i X_{\frac{n}{p}-1}$$

2.7 Analisi Asintotica

In questa sezione viene effettuata l'analisi asintotica dell'algoritmo. Il tempo di esecuzione complessivo può essere diviso in 2 componenti: tempo di calcolo e tempo di comunicazione. Per semplicità si assume che ogni processo ha esattamente $\frac{n}{p}$ elementi.

In particolare il tempo di calcolo è composto da:

-

$$T_{tom_v1} = \Theta\left(\frac{n}{p}\right)$$

Ogni algoritmo applica la variante di Thomas sul proprio blocco di dati di dimensione $\frac{n}{p}$. Quindi vengono effettuate $\frac{n}{p}$ iterazioni nel primo ciclo `for` e $\frac{n}{p} - 2$ nel secondo.

-

$$T_{tom_seq} = \Theta(p)$$

Il processo con rango 0 risolve il sistema ridotto (di dimensione $2p$) con Thomas sequenziale (complessità $\Theta(n)$ su n elementi).

Applicando Thomas sequenziale su $2p$ elementi si ha $T_{tom_seq} = \Theta(2p) = \Theta(p)$

-

$$T_{calcolo_Xi} = \Theta\left(\frac{n}{p}\right)$$

Una volta ricevute le variabili di bordo, ogni processo calcola le sue variabili interne. Il numero di variabili da determinare sono $(\frac{n}{p} - 2)$, pertanto la complessità è pari a $\Theta(\frac{n}{p})$

Il tempo di comunicazione è dato dai seguenti contributi:

-

$$T_{gather} = \Theta(\log p + p) \approx \Theta(p)$$

Utilizzando una struttura di comunicazione ad ipercubo, l'MPI_Gather richiede $\log p$ passi di comunicazioni e manda complessivamente $p - 1$ elementi.

-

$$T_{scatter} = \Theta(\log p + p) \approx \Theta(p)$$

In modo analogo, l'MPI_Scatter può essere implementata utilizzando una struttura di comunicazione ad ipercubo in $\log p$ passi di comunicazione. Anche in questo caso il numero complessivo di elementi trasmessi è $p - 1$.

Dunque, il tempo totale di esecuzione parallelo è:

$$T_p = \underbrace{\Theta\left(\frac{n}{p}\right) + \Theta(p) + \Theta\left(\frac{n}{p}\right)}_{\text{calcolo}} + \underbrace{\Theta(p) + \Theta(p)}_{\text{comunicazione}} \approx \Theta\left(\frac{n}{p}\right) + \Theta(p)$$

2.8 Scalabilità del sistema parallelo

Per quanto detto nella sezione precedente si ha:

- $T_1 = \Theta(n)$
- $T_p = \Theta\left(\frac{n}{p}\right) + \Theta(p)$

Utilizzando la formula $T_o = pT_p - T_1$ si può calcolare l'overhead parallelo:

$$T_o = pT_p - T_1 = n + p^2 - n = \Theta(p^2)$$

Dunque la funzione di isoefficienza è la seguente:

$$n \geq CP^2$$

Questo risultato indica una scarsa scalabilità dell'algoritmo, in quanto per mantenere l'efficienza costante, la problem size deve crescere quadraticamente rispetto al numero di processori utilizzati.

La funzione di occupazione della memoria $M(f(P))$ è lineare rispetto alla dimensione del problema, in quanto è necessario memorizzare esclusivamente i quattro vettori A, B, C, D. Pertanto:

$$\frac{M(f(P))}{P} = \frac{C \cdot P^2}{P} = C \cdot P$$

che rappresenta il limite della scalabilità dal punto di vista della memoria.

2.9 Modello Analitico

Per stimare i parametri di latenza (Λ) e larghezza di banda (β) della macchina utilizzata in fase di benchmark è stato utilizzato il benchmark OMB (OSU Micro-Benchmarks) [2]. Gli OMB sono una suite di test software sviluppata per misurare le performance delle operazioni di comunicazione, con l'obiettivo di valutare metriche come latenza e larghezza di banda.

2.9.1 Latenza (Λ)

Per ricavare il parametro λ è stato utilizzato il benchmark `osu_latency` che esegue una comunicazione di tipo ping-pong tra due processi e riporta la latenza media one-way, cioè la latenza ottenuta dalla media dei tempi relativi all'invio del messaggio solo in fase di andata (per questo one-way). Lanciando il comando `mpirun -np 2 osu_latency` è stato ottenuto l'output riportato in Tabella 1.

Size (B)	Latenza (μs)
0	0.09
1	0.11
2	0.11
4	0.11
8	0.11
16	0.12
32	0.12
64	0.12
128	0.14
256	0.15
512	0.20
1024	0.22
2048	0.29
4096	0.77
8192	1.11
16384	1.42
32768	1.79
65536	2.61
131072	4.56
262144	7.96
524288	18.21
1048576	36.91
2097152	82.72
4194304	186.75

Table 1: Osu_latency Test

Dall'analisi dei dati ottenuti, si è scelto di prendere come valore della latenza

$$\lambda = 0.11 \mu s$$

poiché tale valore è stato misurato per messaggi di dimensione molto piccola. In questo regime, infatti, il tempo di comunicazione ($T_{com} = \lambda + \frac{m}{\beta}$) è dominato dalla latenza, mentre il contributo relativo alla dimensione del messaggio è ancora trascurabile.

2.9.2 Bandwidth (β)

Per ricavare il parametro β è stato utilizzato il benchmark `osu_bw` che misura la bandwidth (Mb/s) per diverse dimensioni dei messaggi inviati. Lanciando il comando `mpirun -np 2 osu_bw` è stato ottenuto l'output riportato in Tabella 2.

Size (B)	Bandwidth (MB/s)
1	15.20
2	32.08
4	63.68
8	128.64
16	244.98
32	489.95
64	726.24
128	1217.24
256	2053.13
512	4861.72
1024	8566.80
2048	13624.95
4096	13443.28
8192	21909.24
16384	29537.35
32768	33182.78
65536	26921.08
131072	22758.02
262144	22124.77
524288	23649.03
1048576	22317.55
2097152	23974.09
4194304	23809.71

Table 2: Osu_bw Test

Dopo un rapido aumento iniziale, la banda tende a stabilizzarsi attorno a una regione quasi costante per messaggi di dimensione elevata. Per questo motivo, come valore per la bandwidth si è scelto di considerare la media degli ultimi sei valori della tabella, ottenendo:

$$\beta = 23624.09 \text{ MB/s}$$

Questa scelta consente di stimare tale valore nella regione in cui il comportamento risulta più stabile.

2.9.3 Modello analitico

Indicando con χ il tempo necessario per l'aggiornamento di un singolo elemento X_i è possibile costruire il seguente modello analitico:

•

$$T_{tom_v1} = 4\chi \left\lceil \frac{n}{p} \right\rceil + 4\chi \left(\left\lceil \frac{n}{p} \right\rceil - 2 \right)$$

Nella funzione `thomas_v1` vengono effettuati $4\frac{n}{p}$ aggiornamenti nel primo ciclo `for` e $4(\frac{n}{p} - 2)$ nel secondo.

-

$$T_{tom_seq} = 3\chi(2p - 1) + \chi 2p$$

Nella versione sequenziale dell'algoritmo di Thomas vengono effettuati $3(2p-1)$ aggiornamenti nel passo di forward e $2p$ nel passo di backward. Si ricorda che l'algoritmo in questione viene utilizzato per risolvere un sistema tridiagonale ridotto di dimensione $2p \times 2p$

-

$$T_{calcolo_Xi} = \chi \left(\left\lceil \frac{n}{p} \right\rceil - 2 \right)$$

Ogni processo dopo aver calcolato le sue variabili di bordo calcola le sue variabili interne effettuando $\frac{n}{p} - 2$ aggiornamenti.

-

$$T_{gather} = \lambda \log p + \frac{8 \cdot 6 \cdot (p - 1)}{\beta}$$

Come discusso in precedenza, utilizzando una struttura di comunicazione ad ipercubo, l'operazione di **Gather** si completa in $\log p$ passi e vengono inviati complessivamente

$$6 \cdot \left(\overbrace{1 + 2 + 2^2 + \dots + 2^{k-1}}^{2^k - 1} \right) = 6(p - 1)$$

elementi.

Il valore 6 deriva dal fatto che ciascun processo invia sei elementi (2 di A, 2 di C e 2 di D). Il fattore 8 tiene conto della dimensione in byte del tipo di dato trasmesso, in questo caso double

- Come discusso nella sezione 2.5 si ha:

$$T_{scatter} = \sum_{i=1}^{\log p} \left(\lambda + 8 \cdot 2p / (2^i \beta) \right) = \lambda \log p + \frac{16(p - 1)}{\beta}$$

Anche in questo caso il fattore 8 indica la dimensione in byte del tipo di dato dell'elemento trasmesso.

Mettendo insieme queste componenti si ottiene il seguente modello analitico:

$$T_p = 4\chi \left\lceil \frac{n}{p} \right\rceil + 4\chi \left(\left\lceil \frac{n}{p} \right\rceil - 2 \right) + 3\chi(2p - 1) + \chi 2p + \chi \left(\left\lceil \frac{n}{p} \right\rceil - 2 \right) + \\ + \lambda \log p + \frac{48(p - 1)}{\beta} + \lambda \log p + \frac{16(p - 1)}{\beta}$$

3 Utilizzo del programma

Per eseguire l'algoritmo, seguire i passaggi riportati di seguito:

1. **Scaricare il repository:** Scaricare il progetto completo dal repository GitHub al seguente link [3].
2. **Compilare il codice:**
Dalla directory principale del progetto lanciare il comando:

```
make
```

3. **Creare la matrice tridiagonale:** Per generare la matrice tridiagonale, eseguire lo script Python:

```
python3 create_tridiagonal_matrix.py n
```

dove n indica la dimensione della matrice da creare.

4. **Eseguire l'algoritmo:** Eseguire il programma parallelo con il comando:

```
mpirun -np P parallel_Thomas A.txt B.txt C.txt D.txt
```

dove P indica il numero di processori da utilizzare.

3.1 Utilizzo osu-micro-benchmarks

Per ricavare i parametri β e λ effettuare i seguenti passi:

1. **Scaricare il repository:** Scaricare il repository dal seguente link
2. **Lanciare il comando:**

make

3. Misura della latenza :

```
mpirun -np 2 ./mpi/pt2pt/osu_latency
```

4. Misura della bandwithdh:

```
mpirun -np 2 ./mpi/pt2pt/osu_bw
```

4 Analisi delle prestazioni e della scalabilità

L'analisi prestazionale è stata eseguita su un MacBook Air (M1, 2020) dotato di un chip Apple M1 che integra una CPU a 8 core (per ulteriori dettagli consultare [1]). I benchmark sono stati eseguiti su una macchina in stato di inattività, cioè con un utilizzo della CPU $\leq 2\%$, al fine di ridurre l'interferenza dovuta ad altri processi in background.

4.1 Input Size (N)

Per effettuare in modo corretto lo studio della scalabilità dell'algoritmo, è stato necessario individuare un valore di N sufficientemente grande da rendere il tempo di esecuzione significativo, ma allo stesso tempo non troppo elevato da eccedere la memoria a disposizione.

Per scegliere in maniera accurata tale valore, è stato eseguito un benchmark variando la dimensione dell'input da 10^2 fino a 10^8 (non sono stati considerati valori superiori per non eccedere la RAM disponibile).

Per ciascun valore di N , l'algoritmo è stato eseguito 10 volte su un singolo processore; per ogni N è stata calcolata la mediana dei tempi ottenuti dalle 10 prove, al fine di ridurre l'influenza di eventuali outlier e ottenere una stima più stabile e rappresentativa del tempo di esecuzione dell'algoritmo. In Tabella 3 sono riportati le mediane dei running time ottenuti al variare di N .

Dall'analisi dei risultati emerge che il valore $N = 10^8$ rappresenta il compromesso migliore in quanto da un lato produce un running time di circa 2 secondi, mentre dall'altro consente al problema di non eccedere la RAM a disposizione.

N	Mediana Tempi di Esecuzione
10^2	0.000004
10^3	0.000016
10^4	0.000138
10^5	0.001350
10^6	0.014228
10^7	0.151747
10^8	1.843503

Table 3: Mediana dei tempi di esecuzione al variare di N

4.2 Modello analitico

Il parametro χ è stato stimato come media aritmetica dei tempi di aggiornamento di un singolo elemento della matrice tridiagonale, misurati su 10 iterazioni consecutive.

Dalle misurazioni sperimentali si è ottenuto:

$$\chi = 1.3 \cdot 10^{-9} \text{ secondi.}$$

Per verificare la correttezza del modello analitico, la cui forma è:

$$T_p = 4\chi \left\lceil \frac{n}{p} \right\rceil + 4\chi \left(\left\lceil \frac{n}{p} \right\rceil - 2 \right) + 3\chi(2p - 1) + \chi 2p + \chi \left(\left\lceil \frac{n}{p} \right\rceil - 2 \right) + \\ + \lambda \log p + \frac{48(p - 1)}{\beta} + \lambda \log p + \frac{16(p - 1)}{\beta}$$

è stato eseguito l'algoritmo di Thomas parallelo con il seguente setup: $p = 4$ e $N = 100000$. Allo stesso modo si è calcolato il risultato teorico atteso sostituendo nel modello analitico i valori di p, N e χ

I risultati ottenuti sono riportati di seguito:

- Modello analitico: 0.00036 secondi
- Tempo di esecuzione reale: 0.00037 secondi

Questo conferma la validità del modello teorico e la corretta implementazione dell'algoritmo.

4.3 Speedup ed efficienza

In questa sezione vengono analizzate le prestazioni del sistema parallelo in termini di speedup ed efficienza.

Lo speedup misura l'accelerazione ottenuta utilizzando p processori rispetto alla versione sequenziale (caso in cui $p = 1$). L'efficienza, invece, misura il grado di utilizzo dei p processori durante l'esecuzione parallela; un'efficienza pari al 100% indica il caso ideale, in cui tutti i processori utilizzano il 100% delle proprie risorse.

Il benchmark è stato eseguito utilizzando i seguenti parametri:

- $N = 100000000$;
- $p = 2^i$ con $i = 0, 1, \dots, 3$.

Per ciascun valore di p sono state eseguite 10 misurazioni indipendenti del tempo di esecuzione della versione parallela dell'algoritmo di Thomas. Tra i valori ottenuti è stata considerata la mediana, in quanto, come già discusso in precedenza, risulta essere meno sensibile alla presenza di outlier.

Le formule utilizzate per il calcolo dello speedup e dell'efficienza sono le seguenti:

Speedup:

$$\psi(p) = \frac{T_1}{T_p}$$

Efficienza:

$$E(p) = \frac{\psi(p)}{p}$$

Nella tabella 4 è possibile visualizzare il tempo di esecuzione reale dell'algoritmo, il tempo ideale (calcolato come $\frac{T_1}{p}$), lo speedup e l'efficienza ottenuti al variare del numero di processori.

P	T_reale (sec)	T_ideale (sec)	Speedup	Efficienza (%)
1	1.904521	1.904521	1.000	100.00
2	1.210676	0.952260	1.573	78.66
4	0.768267	0.476130	2.479	61.97
8	0.632675	0.238065	3.010	37.63

Table 4: Tabella Speedup ed Efficienza

Dai risultati ottenuti si osserva che fino a 2 processori le prestazioni rimangono al di sopra della soglia minima di 70% richiesta per il progetto.

Per $p > 2$, invece, le prestazioni degradano velocemente. Tale comportamento è prevedibile guardando l'analisi di scalabilità precedentemente effettuata, dalla quale emerge che questo algoritmo ha una scalabilità limitata.

La figura 9 mostra l'andamento dello speedup in funzione del numero di processori utilizzati, mettendolo a confronto con lo speedup ideale.

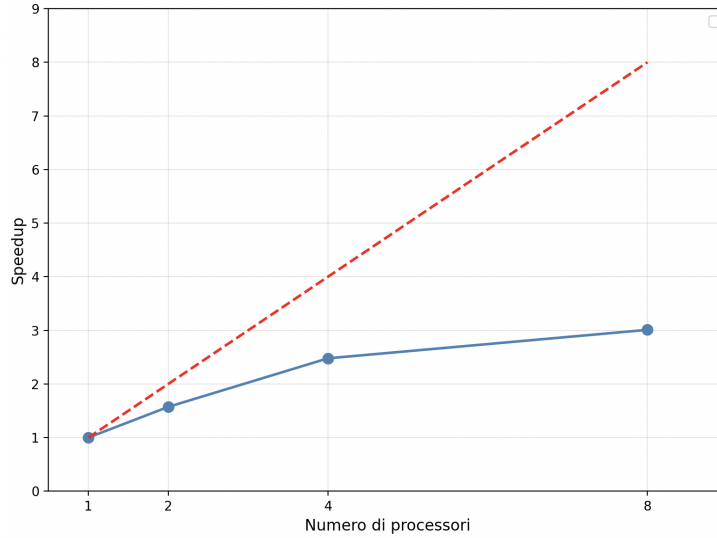


Figure 9: Speedup Vs P

4.4 Strong scalability

In questa sezione viene analizzata la strong scalability della versione parallela dell'algoritmo di Thomas. La strong scalability studia come varia il tempo di esecuzione di un algoritmo al crescere del numero di processori p , mantenendo costante la problem size.

Il benchmark è stato eseguito con i seguenti parametri:

- $N = 100000000$;
- $p = 2^i$ con $i = 0, 1, \dots, 3$.

Nella tabella 5 sono riportati i tempi di esecuzione reale e ideale dell'algoritmo parallelo al variare del numero di processori.

P	T_reale (sec)	T_ideale (sec)	Efficienza (%)
1	1.904521	1.904521	100.00
2	1.210676	0.952260	78.66
4	0.768267	0.476130	61.97
8	0.632675	0.238065	37.63

Table 5: Strong Scalability

I risultati della tabella 5 sono stati rappresentati graficamente in figura 10, dove sull'asse delle ascisse è riportato il numero di processori e sull'asse delle ordinate il tempo di esecuzione, entrambi in scala logaritmica. Nella stessa figura è inoltre mostrato l'andamento dell'efficienza in funzione del numero di processori.

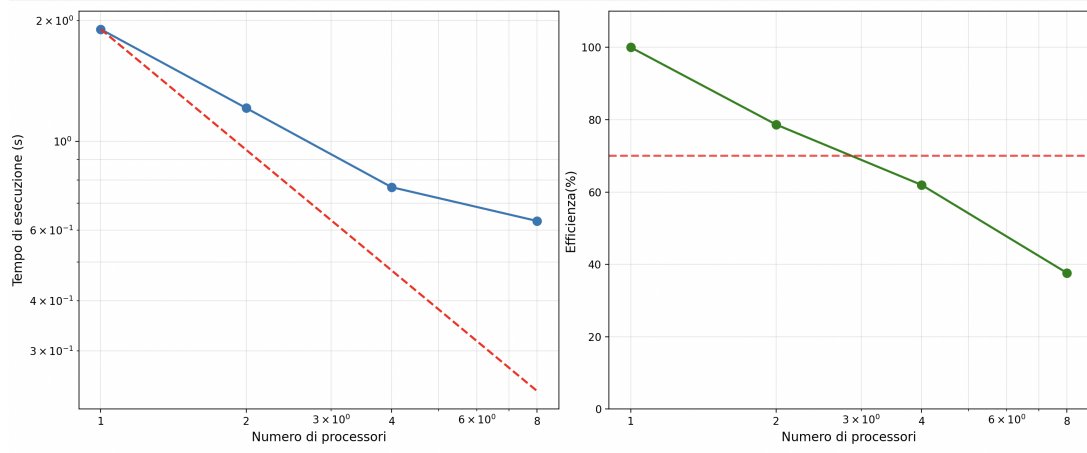


Figure 10: Strong scalability

I risultati mostrano come all'aumentare del numero di processori, la curva in blu relativa ai tempi di esecuzione reali ha una deviazione verso l'alto rispetto alla curva ideale in rosso, indicando una scalabilità limitata dell'algoritmo.

4.5 Weak scalability

In questa sezione viene analizzata la weak scalability del sistema parallelo. La weak scalability valuta la capacità di un algoritmo di mantenere costante il tempo di esecuzione all'aumentare del numero di processori p e della problem size.

Il benchmark è stato eseguito con i seguenti parametri:

- $N = 10000000 \cdot i$ con $i = 1, 2, 4, 8$;
- $p = 2^i$ con $i = 0, 1, \dots, 3$.

Per questa analisi non è stato possibile utilizzare il valore $N = 100000000$, poiché i casi $2N$, $4N$ e $8N$ avrebbero richiesto una quantità di memoria superiore a quella disponibile compromettendo l'affidabilità del benchmark. Anche in questo caso per ogni valore p sono state effettuate 10 misurazioni indipendenti dei tempi di esecuzione prendendo come valore finale la mediana.

Nella tabella 6 sono riportati i tempi di esecuzione reali dell'algoritmo parallelo al variare del numero di processori e della problem size insieme ai valori di speedup scalato ed efficienza scalata. Per il calcolo di speedup ed efficienza sono state utilizzate le seguenti formule:

$$\psi_k = \frac{K \cdot T(1, 1)}{T(p, K)}$$

$$E_k = \frac{\psi_k}{p}$$

dove k rappresenta il fattore di scaling

P	N	T_reale (sec)	ψ_k	E_k
1	10 000 000	0.146618	1.000000	1.000000
2	20 000 000	0.159609	1.837214	0.918607
4	40 000 000	0.208921	2.807144	0.701786
8	80 000 000	0.406114	2.888207	0.361026

Table 6: Weak scalability

I risultati della tabella 6 sono stati rappresentati graficamente in figura 11, dove sull'asse delle ascisse è riportato il numero di processori e sull'asse delle ordinate il tempo di esecuzione, entrambi in scala logaritmica. Nella stessa figura è inoltre mostrato l'andamento dell'efficienza in funzione del numero di processori.

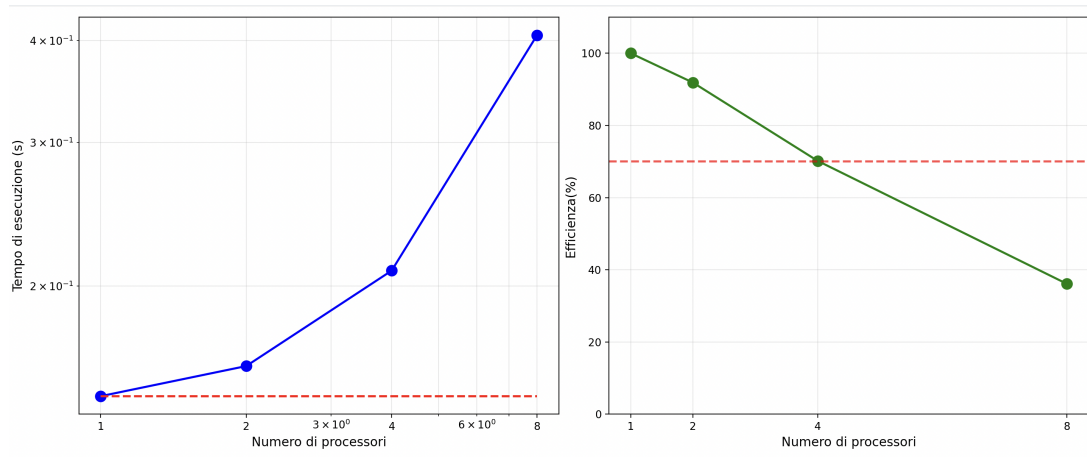


Figure 11: Weak scalability

I risultati mostrano come all'aumentare del numero di processori il tempo di esecuzione non rimane costante ma ha una deviazione verso l'alto rispetto alla curva ideale in rosso; questo evidenzia una scalabilità limitata del nostro

algoritmo dimostrata anche dal secondo grafico presente in figura 11 che mostra una perdita di efficienza all'aumentare del numero di processori e della problem size.

References

- [1] Apple Inc. MacBook Air (M1, 2020) - Specifiche tecniche. <https://support.apple.com/it-it/111883>.
- [2] forresti. OSU Micro-Benchmarks. <https://github.com/forresti/osu-micro-benchmarks>.
- [3] fschirinzi34. Parallel-Thomas-Algorithm. <https://github.com/fschirinzi34/Parallel-Thomas-Algorithm>, 2026.
- [4] Wikipedia. Algoritmo di soluzione dei sistemi tridiagonali, 2024.